Import libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Load Dataset

```
dataset = pd.read_csv('/content/breast-cancer (2).csv')
```

```
print(dataset.head())
```

```
         id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302         M        17.99         10.38          122.80     1001.0
1    842517         M        20.57         17.77          132.90     1326.0
2  84300903         M        19.69         21.25          130.00     1203.0
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   ...  radius_worst  texture_worst  perimeter_worst  area_worst  \
0  ...         25.38          17.33           184.60      2019.0
1  ...         24.99          23.41           158.80      1956.0
2  ...         23.57          25.53           152.50      1709.0
3  ...         14.91          26.50            98.87       567.7
4  ...         22.54          16.67           152.20      1575.0

   smoothness_worst  compactness_worst  concavity_worst  concave points_worst  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   symmetry_worst  fractal_dimension_worst
0          0.4601                  0.11890
1          0.2750                  0.08902
2          0.3613                  0.08758
3          0.6638                  0.17300
4          0.2364                  0.07678

[5 rows x 32 columns]
```

```
type(dataset)
```

```
pandas.core.frame.DataFrame
def __init__(data=None, index: Axes | None=None, columns: Axes | None=None, dtype: Dtype |
None=None, copy: bool | None=None) -> None
```

```
copy : bool or None, default None
    Copy data from inputs.
    For dict data, the default of None behaves like ``copy=True``.  For DataFrame
    or 2d ndarray input, the default of None behaves like ``copy=False``.
    If data is a dict containing one or more Series (possibly of different dtypes),
    ``copy=False`` will ensure that these inputs are not copied.
```

```
dataset.shape
```

```
(569, 32)
```

```
dataset["diagnosis"].unique()
```

```
array(['M', 'B'], dtype=object)
```

```
dataset["diagnosis"].isnull().sum()
```

```
np.int64(0)
```

```
print(dataset["diagnosis"].isna().sum())
```

```
0
```

```
Start coding or generate with AI.
```

Features Engineering

```
dataset["radius_texture"] = (
    dataset["radius_mean"] * dataset["texture_mean"]
)

dataset["area_perimeter_ratio"] = (
    dataset["area_mean"] /
    (dataset["perimeter_mean"] + 1e-10)
)

dataset["radius_ratio"] = (
    dataset["radius_worst"] /
    (dataset["radius_mean"] + 1e-10)
)

dataset["radius_diff"] = (
    dataset["radius_worst"] -
    dataset["radius_mean"]
)

dataset["compactness_concavity"] = (
    dataset["compactness_mean"] *
    dataset["concavity_mean"]
)
```

Define Features & Target

```
X = dataset.drop(['id', 'diagnosis'], axis=1)
Y = dataset['diagnosis']
```

Identify Column Types

```
# Identify Column Types
num_cols = X.select_dtypes(include=np.number).columns
cat_cols = X.select_dtypes(exclude=np.number).columns

print("Numeric columns:", list(num_cols))
print("Categorical columns:", list(cat_cols))
```

```
Numeric columns: ['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavit
Categorical columns: []
```

```
print(dataset["diagnosis"].unique())
```

```
['M' 'B']
```

```
Y = dataset["diagnosis"]

print("Before encoding:")
print(Y.value_counts())

Y = Y.map({"B": 0, "M": 1})
```

```
print("After encoding:")
print(Y.value_counts())
```

```
Before encoding:
diagnosis
B    357
M    212
Name: count, dtype: int64
After encoding:
diagnosis
0    357
1    212
Name: count, dtype: int64
```

```
print(dataset["diagnosis"].unique())
```

```
['M' 'B']
```

Train Test

```
# X_train, X_test, y_train, y_test = train_test_split(
#     X, Y,
#     test_size=0.2,
#     random_state=42,

# )
```

```
# print(y_train.value_counts())
```

```
from imblearn.over_sampling import SMOTE
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42,
    stratify=Y
)
```

```
smote = SMOTE(random_state=42)

X_train, y_train = smote.fit_resample(X_train, y_train)
```

```
print(dataset["diagnosis"].unique())
```

```
['M' 'B']
```

Preprocessing Pipeline

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical processing
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Combine both
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, num_cols),
        ("cat", categorical_transformer, cat_cols)
    ]
)
```

```
# Y = dataset["diagnosis"]

# print("Before encoding:")
# print(Y.value_counts())

# Y = Y.map({"B": 0, "M": 1})

# print("After encoding:")
# print(Y.value_counts())
```

```
print(Y.head())
print(Y.unique())
```

```
0    1
1    1
2    1
3    1
4    1
Name: diagnosis, dtype: int64
[1 0]
```

```
print(Y.head())
print(Y.unique())
```

```
0    1
1    1
2    1
3    1
4    1
Name: diagnosis, dtype: int64
[1 0]
```

Train Test

```
# X_train, X_test, y_train, y_test = train_test_split(
#     X, Y,
#     test_size=0.2,
#     random_state=42,

# )

# from sklearn.model_selection import train_test_split

# X_train, X_test, y_train, y_test = train_test_split(
#     X,
#     Y,
#     test_size=0.2,
#     random_state=42,
#     stratify=Y
# )
```

Apply RF MODEL

```
rf_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestClassifier(n_estimators=200, random_state=42))
])

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print("RANDOM FOREST RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print(confusion_matrix(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))
```

```
RANDOM FOREST RESULTS
Accuracy: 0.9736842105263158
[[72  0]
 [ 3 39]]
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        72
           1       1.00      0.93      0.96        42
```

```
    accuracy                           0.97       114
   macro avg       0.98      0.96      0.97       114
weighted avg       0.97      0.97      0.97       114
```

LR MODEL

```
log_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LogisticRegression(max_iter=1000))
])

log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)

print("LOGISTIC REGRESSION RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_log))
print(confusion_matrix(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))
```

```
LOGISTIC REGRESSION RESULTS
Accuracy: 0.9736842105263158
[[71  1]
 [ 2 40]]
              precision    recall  f1-score   support

           0       0.97      0.99      0.98        72
           1       0.98      0.95      0.96        42

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

XGboost

```
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

xgb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", XGBClassifier(
        n_estimators=200,
        learning_rate=0.1,
        max_depth=6,
        random_state=42,
        eval_metric="logloss"
    ))
])

xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)

print("XGBOOST RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_xgb))
print(confusion_matrix(y_test, y_pred_xgb))
print(classification_report(y_test, y_pred_xgb))
```

```
XGBOOST RESULTS
Accuracy: 0.9824561403508771
[[72  0]
 [ 2 40]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        72
           1       1.00      0.95      0.98        42

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

SVM

```
from sklearn.svm import SVC

svm_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", SVC(kernel="rbf", random_state=42))
])

svm_model.fit(X_train, y_train)

y_pred_svm = svm_model.predict(X_test)

print("SVM RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print(confusion_matrix(y_test, y_pred_svm))
print(classification_report(y_test, y_pred_svm))
```

```
SVM RESULTS
Accuracy: 0.9912280701754386
[[72  0]
 [ 1 41]]
              precision    recall  f1-score   support

           0       0.99      1.00      0.99        72
           1       1.00      0.98      0.99        42

    accuracy                           0.99       114
   macro avg       0.99      0.99      0.99       114
weighted avg       0.99      0.99      0.99       114
```

KNN

```
from sklearn.neighbors import KNeighborsClassifier

knn_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", KNeighborsClassifier(n_neighbors=5))
])

knn_model.fit(X_train, y_train)

y_pred_knn = knn_model.predict(X_test)

print("KNN RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_knn))
print(confusion_matrix(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn))
```

```
KNN RESULTS
Accuracy: 0.9649122807017544
[[71  1]
 [ 3 39]]
              precision    recall  f1-score   support

           0       0.96      0.99      0.97        72
           1       0.97      0.93      0.95        42

    accuracy                           0.96       114
   macro avg       0.97      0.96      0.96       114
weighted avg       0.97      0.96      0.96       114
```

DT

```
from sklearn.tree import DecisionTreeClassifier

dt_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", DecisionTreeClassifier(random_state=42))
])

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)
```

```
print("DECISION TREE RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print(confusion_matrix(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))
```

```
DECISION TREE RESULTS
Accuracy: 0.9298245614035088
[[69  3]
 [ 5 37]]
              precision    recall  f1-score   support

           0       0.93      0.96      0.95        72
           1       0.93      0.88      0.90        42

    accuracy                           0.93       114
   macro avg       0.93      0.92      0.92       114
weighted avg       0.93      0.93      0.93       114
```

Gaussian Naive Bayes

```
from sklearn.naive_bayes import GaussianNB

nb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", GaussianNB())
])

nb_model.fit(X_train, y_train)

y_pred_nb = nb_model.predict(X_test)

print("NAIVE BAYES RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print(confusion_matrix(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb))
```

```
NAIVE BAYES RESULTS
Accuracy: 0.9385964912280702
[[70  2]
 [ 5 37]]
              precision    recall  f1-score   support

           0       0.93      0.97      0.95        72
           1       0.95      0.88      0.91        42

    accuracy                           0.94       114
   macro avg       0.94      0.93      0.93       114
weighted avg       0.94      0.94      0.94       114
```

MLP

```
from sklearn.neural_network import MLPClassifier

mlp_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", MLPClassifier(
        hidden_layer_sizes=(100,),
        max_iter=500,
        random_state=42
    ))
])

mlp_model.fit(X_train, y_train)

y_pred_mlp = mlp_model.predict(X_test)

print("MLP RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_mlp))
print(confusion_matrix(y_test, y_pred_mlp))
print(classification_report(y_test, y_pred_mlp))
```

```
MLP RESULTS
Accuracy: 0.9824561403508771
[[72  0]
```

```
 [ 2 40]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        72
           1       1.00      0.95      0.98        42

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

Extra Trees

```
from sklearn.ensemble import ExtraTreesClassifier

et_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", ExtraTreesClassifier(
        n_estimators=200,
        random_state=42
    ))
])

et_model.fit(X_train, y_train)

y_pred_et = et_model.predict(X_test)

print("EXTRA TREES RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_et))
print(confusion_matrix(y_test, y_pred_et))
print(classification_report(y_test, y_pred_et))
```

```
EXTRA TREES RESULTS
Accuracy: 0.9824561403508771
[[72  0]
 [ 2 40]]
              precision    recall  f1-score   support

           0       0.97      1.00      0.99        72
           1       1.00      0.95      0.98        42

    accuracy                           0.98       114
   macro avg       0.99      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

gb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", GradientBoostingClassifier(
        n_estimators=200,
        learning_rate=0.1,
        random_state=42
    ))
])

gb_model.fit(X_train, y_train)

y_pred_gb = gb_model.predict(X_test)

print("GRADIENT BOOSTING RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_gb))
print(confusion_matrix(y_test, y_pred_gb))
print(classification_report(y_test, y_pred_gb))
```

```
GRADIENT BOOSTING RESULTS
Accuracy: 0.9736842105263158
[[72  0]
 [ 3 39]]
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        72
           1       1.00      0.93      0.96        42
```

```
    accuracy                           0.97       114
   macro avg       0.98      0.96      0.97       114
weighted avg       0.97      0.97      0.97       114
```

AdaBoost

```
from sklearn.ensemble import AdaBoostClassifier

ada_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", AdaBoostClassifier(
        n_estimators=200,
        learning_rate=0.1,
        random_state=42
    ))
])

ada_model.fit(X_train, y_train)

y_pred_ada = ada_model.predict(X_test)

print("ADABOOST RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_ada))
print(confusion_matrix(y_test, y_pred_ada))
print(classification_report(y_test, y_pred_ada))
```

```
ADABOOST RESULTS
Accuracy: 0.9649122807017544
[[72  0]
 [ 4 38]]
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        72
           1       1.00      0.90      0.95        42

    accuracy                           0.96       114
   macro avg       0.97      0.95      0.96       114
weighted avg       0.97      0.96      0.96       114
```

inear Discriminant Analysis (LDA)

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

lda_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", LinearDiscriminantAnalysis())
])

lda_model.fit(X_train, y_train)

y_pred_lda = lda_model.predict(X_test)

print("LDA RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_lda))
print(confusion_matrix(y_test, y_pred_lda))
print(classification_report(y_test, y_pred_lda))
```

```
LDA RESULTS
Accuracy: 0.9736842105263158
[[70  2]
 [ 1 41]]
              precision    recall  f1-score   support

           0       0.99      0.97      0.98        72
           1       0.95      0.98      0.96        42

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

Majority Voting Ensemble

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import VotingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Base Models
lr = LogisticRegression(max_iter=1000, random_state=42)

rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

svm = SVC(
    kernel="rbf",
    probability=True,
    random_state=42
)

# Majority Voting Ensemble
voting_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", VotingClassifier(
        estimators=[
            ("lr", lr),
            ("rf", rf),
            ("svm", svm)
        ],
        voting="hard"   # Majority Voting
    ))
])

# Train
voting_model.fit(X_train, y_train)

# Predict
y_pred_vote = voting_model.predict(X_test)

# Results
print("MAJORITY VOTING ENSEMBLE RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred_vote))
print(confusion_matrix(y_test, y_pred_vote))
print(classification_report(y_test, y_pred_vote))
```

```
MAJORITY VOTING ENSEMBLE RESULTS
Accuracy: 0.9912280701754386
[[72  0]
 [ 1 41]]
              precision    recall  f1-score   support

           0       0.99      1.00      0.99        72
           1       1.00      0.98      0.99        42

    accuracy                           0.99       114
   macro avg       0.99      0.99      0.99       114
weighted avg       0.99      0.99      0.99       114
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, cohen_kappa_score
from sklearn.model_selection import train_test_split
import numpy as np
import pandas as pd

# Assuming X, y are your features and target variable

# Initialize metrics lists to store results for each fold
accuracy_list, precision_list, recall_list, specificity_list, f1_list, kappa_list = [], [], [], [], [], []

for i in range(10):
    X_train, X_test, y_train, y_test = train_test_split(
        X, Y,
        test_size=0.2,
        random_state=i,
        stratify=Y
    )
```

```
        # Create Random Forest classifier
        rf_model = RandomForestClassifier(
            n_estimators=100,
            random_state=i,
            n_jobs=-1
        )

        # Train the Random Forest classifier
        rf_model.fit(X_train, y_train)

        # Make predictions on the test set
        predictions = rf_model.predict(X_test)

        # Evaluate performance metrics
        accuracy = accuracy_score(y_test, predictions) * 100
        precision = precision_score(y_test, predictions, average='macro', zero_division=0) * 100
        recall = recall_score(y_test, predictions, average='macro', zero_division=0) * 100

        cm = confusion_matrix(y_test, predictions)

        if cm.shape == (2, 2):
            tn, fp, fn, tp = cm.ravel()
            specificity = (tn / (tn + fp)) * 100 if (tn + fp) > 0 else 0
        else:
            specificity = np.nan

        f1 = f1_score(y_test, predictions, average='macro', zero_division=0) * 100
        kappa = cohen_kappa_score(y_test, predictions)

        # Append metrics to lists
        accuracy_list.append(accuracy)
        precision_list.append(precision)
        recall_list.append(recall)
        specificity_list.append(specificity)
        f1_list.append(f1)
        kappa_list.append(kappa)

        print("_____")
        print(f'Fold {i + 1}:')
        print(f'Accuracy: {accuracy:.2f}')
        print(f'Precision: {precision:.2f}')
        print(f'Recall: {recall:.2f}')
        print(f'Specificity: {specificity:.2f}' if not np.isnan(specificity) else 'Specificity: N/A')
        print(f'F1 Score: {f1:.2f}')
        print(f'Kappa: {kappa:.4f}')

    # Create DataFrame
    results_df = pd.DataFrame({
        'Fold': np.arange(1, 11),
        'Accuracy': np.round(accuracy_list, 2),
        'Precision': np.round(precision_list, 2),
        'Recall': np.round(recall_list, 2),
        'Specificity': np.round(specificity_list, 2),
        'F1 Score': np.round(f1_list, 2),
        'Kappa': np.round(kappa_list, 4)
    })

    # Add average row
    average_row = {
        'Fold': 'Average',
        'Accuracy': np.round(np.mean(accuracy_list), 2),
        'Precision': np.round(np.mean(precision_list), 2),
        'Recall': np.round(np.mean(recall_list), 2),
        'Specificity': np.round(np.nanmean(specificity_list), 2),
        'F1 Score': np.round(np.mean(f1_list), 2),
        'Kappa': np.round(np.mean(kappa_list), 4)
    }

    results_df = pd.concat([results_df, pd.DataFrame([average_row])], ignore_index=True)

    print("\nFinal Results:")
    print(results_df)
```

```
_____________________________________
Fold 6:
Accuracy: 94.74
Precision: 94.80
Recall: 93.85
Specificity: 97.22
F1 Score: 94.29
Kappa: 0.8858
_____________________________________
Fold 7:
Accuracy: 96.49
Precision: 96.23
Recall: 96.23
Specificity: 97.22
F1 Score: 96.23
Kappa: 0.9246
_____________________________________
Fold 8:
Accuracy: 92.98
Precision: 93.42
Recall: 91.47
Specificity: 97.22
F1 Score: 92.30
Kappa: 0.8462
_____________________________________
Fold 9:
Accuracy: 95.61
Precision: 96.75
Recall: 94.05
Specificity: 100.00
F1 Score: 95.16
Kappa: 0.9034
_____________________________________
Fold 10:
Accuracy: 97.37
Precision: 98.00
Recall: 96.43
Specificity: 100.00
F1 Score: 97.13
Kappa: 0.9426

Final Results:
       Fold  Accuracy  Precision  Recall  Specificity  F1 Score   Kappa
0         1     94.74      93.83   95.34        93.06     94.45  0.8891
1         2     95.61      96.05   94.54        98.61     95.21  0.9043
2         3     96.49      96.23   96.23        97.22     96.23  0.9246
3         4     97.37      96.97   97.42        97.22     97.19  0.9437
4         5     92.98      93.42   91.47        97.22     92.30  0.8462
5         6     94.74      94.80   93.85        97.22     94.29  0.8858
6         7     96.49      96.23   96.23        97.22     96.23  0.9246
7         8     92.98      93.42   91.47        97.22     92.30  0.8462
8         9     95.61      96.75   94.05       100.00     95.16  0.9034
9        10     97.37      98.00   96.43       100.00     97.13  0.9426
10  Average     95.44      95.57   94.70        97.50     95.05  0.9010
```

```
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

svm_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", SVC(kernel="rbf", random_state=42))
])

svm_model.fit(X_train, y_train)

y_pred = svm_model.predict(X_test)

print("SVM RESULTS")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
SVM RESULTS
Accuracy: 0.9649122807017544
[[72  0]
 [ 4 38]]
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        72
           1       1.00      0.90      0.95        42

    accuracy                           0.96       114
```

```
       macro avg       0.97      0.95      0.96       114
    weighted avg       0.97      0.96      0.96       114
```

```
Start coding or generate with AI.
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.